

# **Алгоритмы и Структуры Данных**

**Автор: Рязских Дмитрий**

**Лектор: Кулапин Артур Евгеньевич**

# Содержание

- 1) Основные понятия
- 2) Линейные структуры данных
- 3) Древовидные структуры данных
- 4) Сортировки
- 5) Динамическое программирование
- 6) Алгоритмы на графах
  - 6.1) Обходы графов
    - 6.1.1) DFS
    - 6.1.2) Лемма о белых путях
    - 6.1.3) Топологическая сортировка графа
    - 6.1.4) Мосты и точки сочленения в графе
    - 6.1.5) Конденсация графа
    - 6.1.6) Алгоритм Косарайю
    - 6.1.7) Проверка графа на связность
    - 6.1.8) Проверка графа на ацикличность
    - 6.1.9) BFS
    - 6.1.10) 0-k BFS
  - 6.2) Поиск путей в графах
    - 6.2.1) Алгоритм Дейкстры
    - 6.2.2) Эвристики
      - 6.2.2.1) Алгоритм A\*
      - 6.2.2.2) Алгоритм IDA\*
    - 6.2.3) Алгоритм Форда-Беллмана
    - 6.2.4) SPFA
    - 6.2.5) Алгоритм Флойда-Уоршелла
    - 6.2.6) Циклы отрицательного веса
    - 6.2.7) Алгоритм Джонсона
    - 6.2.8) Двусторонние алгоритмы
  - 6.3) CHM и LCA
  - 6.4) Минимальное остовное дерево
  - 6.5) Алгоритмы в двудольных графах
  - 6.6) Сети в графах
- 7) Геометрическое программирование

## Базовый минимум пройденных курсов

Прежде чем начать читать этот конспект, настоятельно рекомендуется пройти следующие курсы:

- 1. Введение в программирование (любой ЯП)
- 2. Дискретная математика
- 3. Математическая логика

# Алгоритмы на графах

## Обходы графов

### DFS

**Опр.** **Depth First Search** (*DFS*, *обход в глубину*) - обход графа, в ходе которого вершины посещаются сразу при их обнаружении.

#### Псевдокод

```
def dfs(G):
    for v in G.V:
        color[v] = "white"

    time = 0
    for v in G.V:
        if color[v] == "white":
            dfs_visit(G, v)

def dfs_visit(G, u):
    global time

    time += 1
    t_in[u] = time
    color[u] = "gray"

    for v in G.adj[u]:
        if color[v] == "white":
            dfs_visit(G, v)

    color[u] = "black"
    time += 1
    t_out[u] = time
```

#### Раскраска DFS

Состояние  $\forall v \in V$  описывается парой  $(c, t)$ , где  $c$  - её фактический цвет,  $t$  - значение таймера вершины.

Выделяются три цвета раскраски:

- **Белый** - вершина ещё не посещена обходом.
- **Серый** - вершина посещена, но рекурсия ещё не вышла из неё.
- **Чёрный** - рекурсия вышла из вершины.

#### Неорграф

В неориентированном графе серый цвет не является обязательным.

Таймер фиксирует время входа и выхода из вершины:

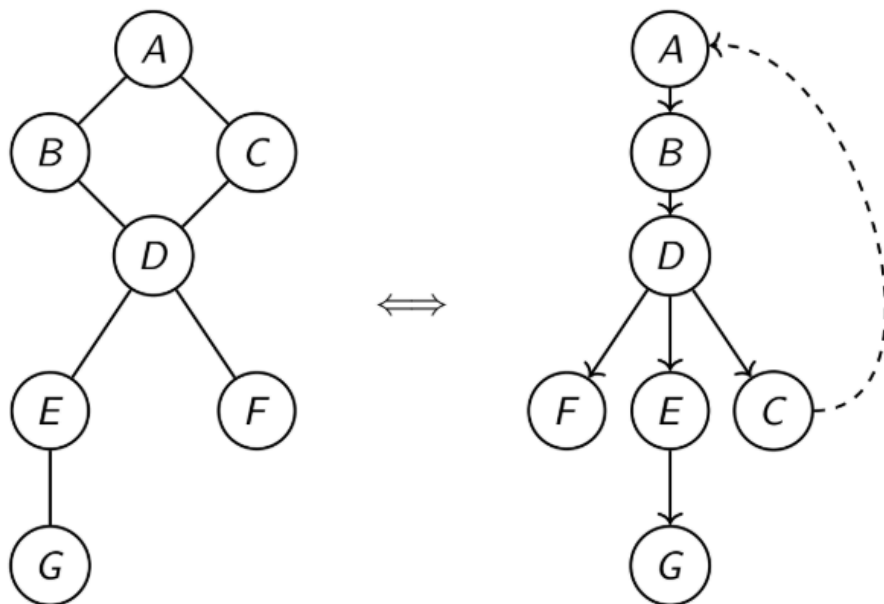
- Время входа  $t_{in}(v)$  - такт, когда вершина  $v$  окрашивается в серый цвет.
- Время выхода  $t_{out}(v)$  - такт, когда вершина  $v$  окрашивается в чёрный цвет.

#### Дерево обхода

**Опр. Древесное ребро** - ориентированное ребро, по которому *DFS* переходит напрямую (переход из серой вершины в белую).

**Опр. Обратное ребро** - ориентированное ребро, которое *DFS* просматривает, но не идёт по нему (переход в серую вершину).

**Опр. Дерево обхода DFS** - граф, состоящий из вершин, посещаемых в ходе обхода *DFS*, а также соответствующих древесных и обратных рёбер.



На рисунке слева представлен исходный граф, справа - дерево обхода. Древесные рёбра показаны сплошной линией, обратные - пунктирной.

**Опр. Белый путь** - путь в дереве обхода DFS, все вершины в котором *(быть может кроме первой)* имеют белый цвет.

## Асимптотика

Для сильно связного графа справедливы следующие свойства:

1. Сложность по времени:  $O(|V| + |E|)$ , поскольку  $\forall v \in V$  посещается ровно один раз, при этом просматриваются все её смежные рёбра.
2. Сложность по памяти:  $O(|V|)$ , поскольку максимальный объём памяти ограничен глубиной стека рекурсии  $\Rightarrow$  в стеке одновременно находится не более  $|V|$  элементов.



## Лемма о белых путях

Пусть  $v \in V$ , тогда  $\forall u \in V$  достижимой по белым путям из  $v$  в момент  $t_{in}(v)$  : к моменту  $t_{out}(v)$  цвет вершины  $u$  станет чёрным.

### Доказательство

Докажем по индукции по убыванию значений  $t_{in}$ .

**База:** Пусть  $u \in V$  имеет максимальное значение  $t_{in}(u) \Rightarrow$  из  $u$  отсутствуют белые пути. (иначе  $\exists w \in V : t_{in}(w) > t_{in}(u)$ , что противоречит максимальнойности).

**Шаг:** Предположим противное. Пусть  $u$  - первая в порядке обхода вершина, цвет которой  $\neq$  чёрный в момент  $t_{out}(v)$ .

1. Предок вершины  $u$  на белом пути является чёрным по предположению  $\Rightarrow$  все её дочерние вершины уже полностью обошли  $\Rightarrow u$  не может быть белой.
2. Предположим  $u$  - серая, тогда  $t_{in}(v) < t_{in}(u) < [\text{т.к. } u \text{ ещё не обработана}] < t_{out}(v) < t_{out}(u)$ . Полученная цепочка неравенств напрямую противоречит структуре стека рекурсии  $DFS$  (правило LIFO)  $\Rightarrow$  предположение неверно.



## Топологическая сортировка графа

Пусть  $G$  - ориентированный граф,  $\sigma : V \rightarrow \mathbb{N}$  - нумерованная перестановка вершин.

**Опр.** Топологическая сортировка (*topsort*) - перестановка  $\sigma$ , т.ч.  $\forall (u, v) \in E : \sigma(u) < \sigma(v)$ .

### Критерий существования топологической сортировки

- $\exists$  топологическая сортировка в  $G \iff$  граф  $G$  ацикличен.

#### Доказательство

Корректность задания порядка  $\sigma$  напрямую следует из линейности отношения порядка сравнения и конечности множества  $V$ .

### Построение topsort

Алгоритм построения:

1. Запуск DFS на каждом компоненте связности графа  $G$ .
2. В момент  $t_{out}(v)$  - добавление вершины  $v$  в нелокальный массив  $arr$ .
3. Инвертирование полученного массива  $\Rightarrow arr$  - искомый topsort.

#### Корректность

В  $DFS$  : если  $u$  достижима из  $v$ , то  $t_{out}(v) > t_{out}(u)$ . Таким образом, искомая топологическая сортировка - это сортировка в порядке убывания времён выхода.

#### Асимптотика

1. Сложность по времени:  $O(|V| + |E|)$  - эквивалентно обходу в глубину.
2. Сложность по памяти:  $O(|V|)$  - стек  $DFS$  + хранение массива результатов.



## Мосты и точки сочленения в графе

**Опр.**  $t_{up}(v)$  - функция вида:

$$t_{up}(v) = \min\{t_{in}(v), \min_u t_{in}(u)\}$$

где  $u$  - предок вершины  $v$ , достижимый из поддеревя обхода  $v$ . Функция задаёт минимальное время входа среди всех таких вершин.

Подсчёт  $t_{up}$

```

void FillTUp(Graph graph, Vertex from, Vertex ancestor) {
    visited[from] = true;
    t_in[from] = t_up[from] = timer++;
    for (Vertex to : graph.GetNeighbours(from)) {
        if (to == ancestor) { continue; }
        if (visited[to]) {
            t_up[from] = min(t_up[from], t_in[to]);
        } else {
            FillTUp(graph, to, from);
            t_up[from] = min(t_up[from], t_up[to]);
        }
    }
}

```

### Критерий моста

- Ребро  $(t, v)$  является мостом  $\iff t_{up}(v) = t_{in}(v)$ .

#### Доказательство

1. По определению:  $t_{up}(v) \leq t_{in}(v)$ .
2. Равенство  $t_{up}(v) = t_{in}(v)$  достигается  $\iff \nexists u \in V$  : из поддерева  $v$  ведёт ребро в предка вершины  $v$  в дереве обхода. Данное условие эквивалентно определению моста.

### Критерий точки сочленения

1. Если  $v$  - корень дерева обхода, то  $v$  - точка сочленения  $\iff$  у неё количество дочерних вершин  $> 1$ .
2. Если  $v$  - не корень дерева обхода, то  $v$  - точка сочленения  $\iff t_{up}(v) = t_{in}(v)$ .

#### Доказательство

1.  $v$  - корень дерева обхода.

**Необходимость:** Если у  $v$  ровно 1 дочерняя вершина  $\Rightarrow$  её удаление не нарушает связность оставшегося поддерева.

**Достаточность:** При полном обходе одного поддерева остальные поддеревья остаются белыми  $\Rightarrow$  единственный путь между ними лежит через корень  $\Rightarrow$  удаление  $v$  разбивает граф.

2.  $v$  - не корень дерева обхода.

**Необходимость:** Предположим противное:  $t_{up}(v) < t_{in}(v) \Rightarrow \exists$  путь в предка  $v$  в обход самой  $v \Rightarrow v$  не является точкой сочленения, противоречие.

**Достаточность:**  $t_{up}(v) = t_{in}(v) \Rightarrow \nexists$  пути выше  $v$  из её поддеревьев  $\Rightarrow$  при удалении  $v$  её потомки изолируются от предков.



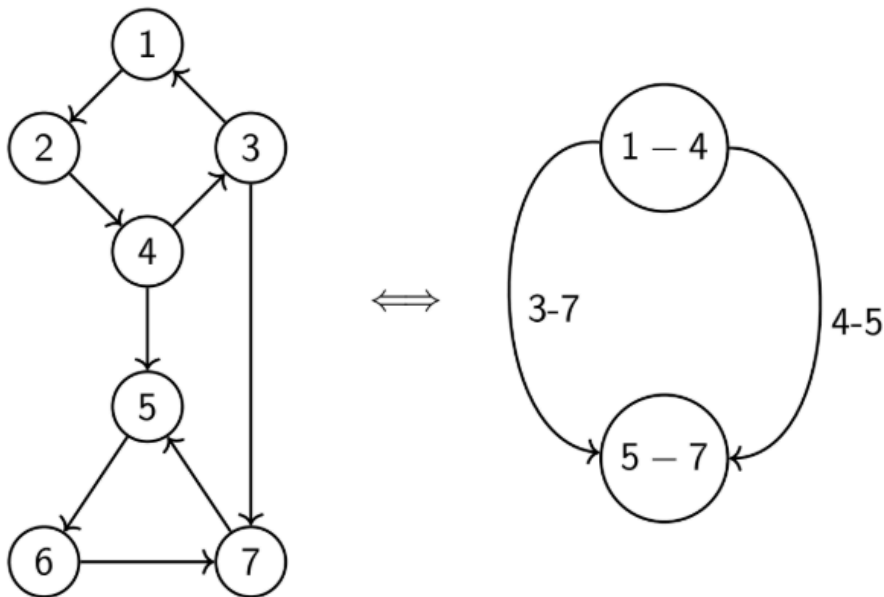
### Конденсация графа

**Опр. Граф конденсации** (графа  $G$ ) - граф, в котором вершинами являются классы эквивалентности вершин исходного графа, а рёбра задают отношения между ними.

Примеры отношений эквивалентности:

- Сильная связность между вершинами (рёбра конденсации - рёбра между компонентами сильной связности).
- Двусвязность между вершинами (рёбра конденсации - мосты).

### Ацикличность графа конденсации сильной связности



- Граф конденсации по КСС является ациклическим.

### Доказательство

Предположим противное:  $\exists$  цикл из компонент  $C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_k \rightarrow C_1$ . По определению конденсации :

$\forall i \in \overline{1, k-1} \exists$  путь из  $C_i$  в  $C_{i+1}$ , а также  $\exists$  путь из  $C_k$  в  $C_1 \Rightarrow \forall u_i \in C_i$  и  $\forall v_j \in C_j$  выполняются условия взаимной достижимости  $\Rightarrow$  Все вершины  $\bigcup_{i=1}^k C_i$  принадлежат одной КСС, а не разным.



### Алгоритм Косарайю

#### Алгоритм

1. Запуск DFS на графе  $G$  для вычисления времён выхода  $t_{out}(v)$ .
2. Построение транспонированного графа  $G^T$ .
3. Запуск DFS на графе  $G^T$  в порядке убывания значений  $t_{out}$ .
4. Каждая найденная компонента связности формирует отдельную КСС исходного графа.

#### Псевдокод

```

HashMap<Vertex, int> Cosaraju(Graph graph) {
    Array<Vertex> second_phase_starts; // ascending by t_out
    DFS(graph, graph.GetAnyVertex(), second_phase_starts);
    Graph transposed = graph.GetTransposed();
    HashMap<Vertex, int> scc2color;
    int color = 0;
    while (scc2color.size() < graph.GetVerticesCount()) {
        // inserts all visited vertices to scc2color
        DFS(transposed, scc2color.FirstNotFound(), scc2color,
            color++);
    }
}

```

## Корректность

### Лемма о временах выхода

Пусть  $C, C'$  - различные вершины графа конденсации, т.ч.  $\exists(C, C') \in E_{\text{конд}}$ , тогда  $t_{\text{out}}(C) > t_{\text{out}}(C')$ , где:

$$t_{\text{out}}(C) = \max_{v \in C} t_{\text{out}}(v)$$

$$t_{\text{in}}(C) = \min_{v \in C} t_{\text{in}}(v)$$

### Доказательство

1. Если  $t_{\text{in}}(C) < t_{\text{in}}(C')$ , то DFS зайдёт в  $C$  раньше. По лемме о белых путях все вершины  $C'$  будут полностью посещены до момента  $t_{\text{out}}(C) \Rightarrow t_{\text{out}}(C) > t_{\text{out}}(C')$ .
2. Если  $t_{\text{in}}(C) > t_{\text{in}}(C')$ , то DFS зайдёт в  $C'$  раньше. Из  $C'$  нет путей в  $C$  по свойствам графа конденсации  $\Rightarrow C'$  полностью обработается до перехода в  $C \Rightarrow t_{\text{out}}(C) > t_{\text{out}}(C')$ .

## Корректность Косарайю

Достаточно доказать, что на шаге 3 алгоритма Косарайю каждый запуск DFS на  $G^T$  посещает ровно одну КСС.

### Доказательство

Докажем по индукции по убыванию значений  $t_{\text{out}}(C)$ .

**База:** Для компоненты с  $\max_C t_{\text{out}}(C)$  отсутствуют входящие рёбра в  $G$ . В  $G^T$  из неё отсутствуют исходящие рёбра  $\Rightarrow$  DFS посетит исключительно вершины этой КСС.

**Шаг:** Те КСС, которые уже рассмотрены, имеют время выхода раньше, поэтому посещены DFS и все их вершины покрашены в чёрный цвет, а значит в обходе  $G^T$  в данном запуске мы их посетить не можем и текущая КСС максимальная по времени выхода из оставшихся (сработала база индукции).

### Свойство Косарайю

Получили, что алгоритм генерирует КСС, причём в порядке топологической сортировки, так как он работает по убыванию времени выхода.

## Асимптотика



1. По времени:

DFS по двум равноразмерным графам ( $G$  и  $G^T$ ) =  $2 \cdot O(|V| + |E|)$

Постройка транспонированного графа:  $O(|V| + |E|)$

**Итоговая асимптотика:**  $O(|V| + |E|)$

2. По памяти:

Требуется хранение транспонированного графа + стек dfs.

**Итог:**  $O(|V| + ||G||)$



## Проверка графа на связность

### Критерий слабой связности

- Орграф  $G$  слабо связан  $\iff$  его сопряжённый неорграф сильно связан  
Это эквивалентно:
- $G = (V, E)$  - слабо связан  $\iff \forall u, v \in G^* = (V, E \cup E^T) : \exists p (в G^*) = \{u \rightarrow \dots \rightarrow v\}$

**Алгоритм проверки:**

1. Запуск DFS от произвольной вершины  $s$  на сопряжённом неориентированном графе.
2. По лемме о белых путях: все вершины должны окраситься в чёрный цвет.
3. Проверка равенства числа посещённых вершин значению  $|V|$ .



## Проверка графа на ацикличность

### Критерий ацикличности

- $\exists$  ориентированный цикл, достижимый из  $s \iff$  DFS из  $s$  обнаруживает ребро в серую вершину.

**Доказательство**

**Необходимость:**

Пусть  $C$  - цикл, достижимый из  $s$ . Пусть  $v \in C$  - первая посещённая вершина цикла,  $(u, v) \in C$ . В момент  $t_{in}(v)$  весь цикл белый  $\Rightarrow$  по лемме о белых путях к моменту  $t_{out}(v)$  весь цикл станет чёрным. При обработке вершины  $u$  цвет вершины  $v$  является серым  $\Rightarrow$  обнаружено ребро в серую вершину.

**Достаточность:**

Стек рекурсии представляет собой путь, состоящий исключительно из серых вершин. Нахождение ребра в серую вершину означает замыкание пути на элемент выше по стеку  $\Rightarrow \exists$  ориентированный цикл.



## BFS

**Опр.** **Breadth-First Search (BFS, обход в ширину)** - обход графа, в ходе которого вершины посещаются по уровням.

**Псевдокод**

## Псевдокод

```
void BFS(Graph graph, Vertex start) {
    Queue<Vertex> bfs_queue;
    HashMap<Vertex, int> dist;
    dist[start] = 0;
    bfs_queue.push(start);
    while (!bfs_queue.empty()) {
        Vertex from = bfs_queue.pop();
        for (Vertex to : graph.GetNeighbours(from)) {
            if (dist.HasKey(to)) { continue; }
            dist[to] = dist[from] + 1;
            bfs_queue.push(to);
        }
    }
}
```

### 📌 Вид обхода

Обход "волнообразный": если все рёбра невзвешенные, то сначала посещаются все вершины на расстоянии 1 от стартовой, затем на расстоянии 2 и т.д. по возрастанию. Такой обход обеспечивается свойствами очереди.



### 0-k BFS

### 0-1 BFS

Пусть  $G$  - взвешенный граф, в котором рёбра имеют вес только 0 или 1. Требуется найти кратчайшие расстояния от заданной стартовой вершины  $s$  до всех остальных вершин графа.

### Алгоритм

Модификация обхода в ширину, где вместо очереди используется дека:

- Если вес рассматриваемого ребра равен 0  $\Rightarrow$  добавление вершины в начало дека.
- Иначе  $\Rightarrow$  в конец дека.

Данный алгоритм строит уровни эквивалентности при нулевых переходах (если  $v$  и  $u$  связаны путём нулевого веса, следовательно они находятся в одном классе эквивалентности).

### Корректность

#### 🔗 Инвариант

В деке всегда хранятся вершины, упорядоченные по неубыванию расстояний от стартовой вершины  $s$ .

### Доказательство

1. На каждом шаге извлекается вершина с минимальным текущим расстоянием из начала дека.
2. При переходе по рёбрам: нулевые рёбра не увеличивают расстояние  $\Rightarrow$  вершины добавляются в начало дека, сохраняя порядок.
3. Рёбра веса 1 увеличивают расстояние на 1  $\Rightarrow$  вершины добавляются в конец дека, что сохраняет упорядоченность по неубыванию.

**Следствие:** первое посещение вершины гарантированно даёт кратчайшее расстояние.

### Асимптотика

- **Сложность по времени:**  $O(|V| + |E|)$ , т.к.  $\forall v \in V$  и  $\forall e \in E$  обрабатываются ровно один раз, а операции с деком выполняются за  $O(1)$ .
- **Сложность по памяти:**  $O(|V|)$ , т.к. дек может содержать до  $|V|$  вершин.

## 1-k BFS

Пусть  $G$  - взвешенный граф, где веса рёбер принимают целочисленные значения от 1 до  $k$ . Требуется найти кратчайшие расстояния от заданной стартовой вершины  $s$  до всех остальных вершин графа.

### Алгоритм

Модификация BFS с  $k + 1$  очередями:

1. При обработке рёбер текущее расстояние складывается с весом ребра (суммарный вес  $x$ ).
2. Вершина добавляется в  $(x \bmod (k + 1))$ -ю очередь.
3. На каждом шаге обрабатываются вершины из очереди, соответствующей текущему расстоянию по модулю  $k + 1$ .

### Корректность

#### Инвариант

Вершины в очередях упорядочены по неубыванию расстояний.

### Доказательство

1. Очередь  $i$  содержит вершины с расстоянием  $d \equiv i \pmod{k + 1}$ .
2. Т.к. максимальный вес ребра равен  $k \Rightarrow$  разница между соседними расстояниями не превышает  $k$ .
3. Циклический обход очередей гарантирует, что вершина будет обработана при минимально возможном расстоянии.

**Следствие:** каждая вершина обрабатывается ровно один раз с минимальным расстоянием.

### Асимптотика

- **Сложность по времени:**  $O(k|V| + |E|)$ , т.к. каждая вершина может быть обработана до  $k$  раз (по разным очередям), а каждое ребро обрабатывается один раз. Операции с очередями выполняются за  $O(1)$ .
- **Сложность по памяти:**  $O(k|V|)$  для хранения  $k + 1$  очередей и массива расстояний.

## 0-k BFS

Пусть  $G$  - взвешенный граф, где веса рёбер принимают целочисленные значения от 0 до  $k$ . Требуется найти кратчайшие расстояния от заданной стартовой вершины  $s$  до всех остальных вершин графа.

### Алгоритм

Комбинация подходов 0-1 BFS и 1-k BFS:

- Создаётся  $k + 1$  деков, т.ч. при встрече ребра нулевого веса вершина добавляется в начало соответствующей очереди.



## Поиск путей в графах

### Алгоритм Дейкстры

#### Цель

Алгоритм находит кратчайшее расстояние между двумя вершинами графа или от одной вершины ко всем при условии неотрицательных весов рёбер.

**Опр. Релаксация** - обновление расстояния до вершины, если найден более короткий путь через другую вершину.

#### Алгоритм

Пусть  $s$  - начальная вершина,  $S$  - множество вершин, для которых кратчайшее расстояние вычислено корректно (изначально  $S = \emptyset$ ). Пусть  $d[v]$  - оценка сверху на кратчайшее расстояние между  $s$  и  $v$ , т.ч.  $d[s] = 0$ , а для остальных  $d[v] = +\infty$ .

1. Пока  $S \neq V$ :

1. Выбирается вершина  $from \in V/S$ , т.ч.  $d[from] = \min_{v \notin S} d[v]$ .
2.  $S = S \cup \{from\}$ .
3. Кратчайшее расстояние до  $from$  найдено и равно  $d[from]$  (На этом шаге можно оборвать алгоритм если нужна была только вершина  $from$ )
4.  $\forall (from, to, weight) \in E$ : проведём релаксацию:

$$d[to] = \min\{d[to], d[from] + weight\}$$

2. В массиве  $d$  лежат минимальные расстояния.

#### Корректность

Докажем по индукции по  $S$ .

**База:** На первой итерации  $S = \{s\}$  - расстояние нулевое, всё корректно.

**Предположение:** корректно посчитано расстояние до вершины  $u$  ( $u \in S$ ).

**Шаг:** Пусть до вершины  $v$ , которую мы добавляем в  $S$  (следовательно,  $d[v] = \min_{t \notin S} d[t]$ ), была проведена релаксация через вершину  $u$ , то есть  $d[v] = d[u] + weight_{u \rightarrow v}$ .

Рассмотрим реальный кратчайший путь от  $s$  до  $v$ .

1. Путь содержит вершину  $u$

1. Путь содержит ребро  $(u, v) \Rightarrow$  его длина не меньше  $d[v]$  (т.к.  $d[u]$  - кратчайшее до  $u \Rightarrow d[v]$  - кратчайший путь при таких условиях).
2. Путь не содержит ребро  $(u, v) \Rightarrow \exists t \notin S : d[t] \leq d[v]$  и  $s \rightsquigarrow u \rightsquigarrow t \rightsquigarrow v$ .
  1. Если  $d[t] = d[v] \Rightarrow$  путь не короче найденного ( $weight_{t \rightsquigarrow v} \geq 0$ ),
  2. Если  $d[t] < d[v] \Rightarrow$  противоречие с минимальностью  $v$ .

2. Путь не содержит вершину  $u$ , тогда:

$\exists m \in S \exists t \notin S : d[t] = d[m] + weight_{m \rightarrow t} \leq d[v]$  - в случае равенства путь получится не короче найденного ( $weight_{t \rightsquigarrow v} \geq 0$ ), а в случае знака меньше получим противоречие минимальности  $v$ .

Таким образом, только в случае 1.1 получился достоверно корректный минимальный путь и он совпадает с  $d[v]$ .

## Асимптотика

Сложность по времени складывается из следующих операций:

- Уменьшение оценки  $d$  для вершины: каждое ребро участвует в релаксации, значит таких операций  $O(|E|)$
- Извлечение вершины с минимальной оценкой из  $V \setminus S : \forall v \in V$  извлекается не более одного раза  $\Rightarrow O(|V|)$  операций.
- Время выполнения зависит от реализации множества  $S$ :

| Реализация $S$   | Релаксация      | Извлечение      | Итог                          |
|------------------|-----------------|-----------------|-------------------------------|
| Массив           | $O(1)$          | $O(\ V\ )$      | $O(\ E\  \ V\ ^2)$            |
| Дерево поиска    | $O(\log \ V\ )$ | $O(\log \ V\ )$ | $O(\ E\  \log \ V\ )$         |
| Фибоначиева куча | $O(1^*)$        | $O(\log \ V\ )$ | $O(\ E\  + \ V\  \log \ V\ )$ |



## Эвристики

**Опр. Эвристика** - идея оценки результата, основанная на свойствах предметной области или отдельной конкретной задачи.

**Опр. Эвристический алгоритм** - алгоритм, использующий эвристики.

Рассмотрим эвристические алгоритмы поиска путей в графах. Пусть  $h(v)$  - эвристика,  $dist(u, v)$  - кратчайшее расстояние от  $u$  до  $v$ , тогда при поиске кратчайшего расстояния от  $s$  до любой другой вершины:

**Опр. Допустимая эвристика** - эвристика, т.ч.  $\forall v \in V : h(v) \leq dist(s, v)$ .

**Опр. Монотонная эвристика** - эвристика, т.ч.  $\forall (u, v) \in E : h(v) - h(u) \leq weight_{u \rightarrow v}$  и  $h(s) = 0$ .

### Связь монотонной и допустимой эвристик

- Любая монотонная эвристика допустима.

#### Доказательство

Докажем по индукции.

**База:**  $h(s) = 0 \leq 0 = dist(s, s)$ .

**Предположение:**  $\forall u \in V$  (при  $dist(s, u) = k$ ) :  $h(u) \leq dist(s, u)$ .

**Шаг:** Пусть вершина  $v$  такая, что  $dist(s, v) = \min\{dist(s, t) \mid t \in V \wedge dist(s, t) > k\}$ . Пусть  $u$  - вершина на кратчайшем пути от  $s$  до  $v$  и ребро  $(u, v)$  - часть этого пути, тогда т.к.  $h(v) - h(u) \leq weight_{u \rightarrow v}$ , то выполняется:

$$h(v) \leq h(u) + weight_{u \rightarrow v} \leq weight_{u \rightarrow v} + dist(s, u) = dist(s, v)$$



## Алгоритм A\*

### Цель

Алгоритм  $A^*$  - это эвристический алгоритм Дейкстры.

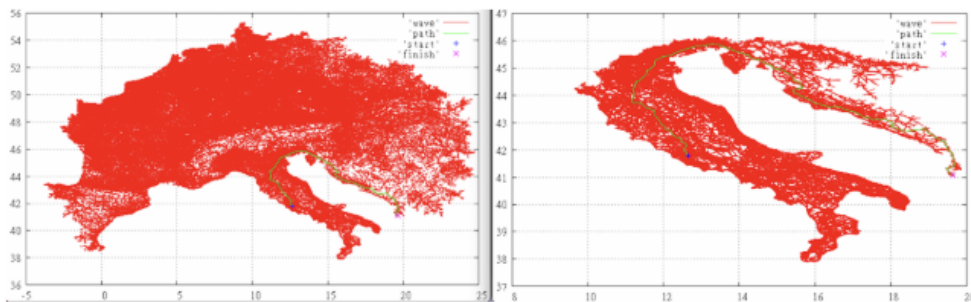
В алгоритме  $A^*$  вместо минимума по расстоянию берётся минимум по  $d[v] + h(v)$ , где  $h$  - эвристическая функция.

### Свойство

В случае монотонной эвристики алгоритм  $A^*$  работает не хуже алгоритма Дейкстры.

## Распространённые эвристики

### 1. Евклидово расстояние



На рисунке показаны вершины, рассмотренные алгоритмом поиска пути по автомобильным дорогам из Рима в Тирану: слева - алгоритм Дейкстры, справа -  $A^*$  с евклидовой эвристикой.



## Алгоритм IDA\*

## Алгоритм Форда-Беллмана

### Цель

Алгоритм находит кратчайшее расстояние в графе от одной вершины ко всем при условии отсутствия циклов отрицательного веса.

## Алгоритм

В основе работы лежит принцип динамического программирования.

Пусть  $dp[v][k]$  - минимальный вес пути из  $s$  в  $v$ , состоящего ровно из  $k$  рёбер.

1. База:  $dp[s][0] = 0$ , для остальных вершин выполняется:  $dp[v][0] = +\infty$ .

2. Переход:

$$dp[v][k] = \min_{(u,v) \in E} (dp[u][k-1] + weight_{u \rightarrow v})$$

3. Порядок пересчёта: внешний цикл по  $k$ , внутренний - по рёбрам графа.

4. Ответ:  $\forall v \in V : \min_k dp[v][k]$ .

## Корректность

Докажем по индукции по параметру  $k$ .

**База:**  $k = 0$ . Путь из  $s$  в  $s$  из 0 рёбер имеет вес 0:  $dp[s][0] = 0$ .  $\forall v \in V / \{s\}$  путей из 0 рёбер не существует  $\Rightarrow dp[v][0] = +\infty$ .

**Предположение:**  $\forall u \in V$  значение  $dp[u][k-1]$  корректно задаёт минимальный вес пути из  $s$  в  $u$ , состоящего ровно из  $k-1$  рёбер.

**Шаг:** Любой путь из  $s$  в  $v$ , состоящий ровно из  $k$  рёбер, представляет собой путь длины  $k-1$  до некоторого предка  $u$  и ребро  $(u, v) \in E$ . По предположению индукции, минимальный вес пути до вершины  $u$  равен  $dp[u][k-1] \Rightarrow$  минимальный вес пути до  $v$  через конкретного предка  $u$  равен  $dp[u][k-1] + weight_{u \rightarrow v}$ . Для нахождения глобального

минимума необходимо взять минимум по всем возможным предшественникам:

$$dp[v][k] = \min_{(u,v) \in E} (dp[u][k-1] + weight_{u \rightarrow v})$$

Т.к. в графе отсутствуют циклы отрицательного веса, кратчайший простой путь  $\forall v \in V$  содержит в себе не более  $|V| - 1$  рёбер  $\Rightarrow$  итоговое кратчайшее расстояние гарантированно находится как  $\min_k dp[v][k]$ .

## Асимптотика

- Сложность по времени:  $O(|V| \cdot |E|)$ . На каждом шаге алгоритма полностью перебираются все рёбра. Максимальная длина простого пути равна  $|V| \Rightarrow$  выполняется  $\max_k k = |V|$ , из чего следует общее число шагов равно  $|V|$ .
- Сложность по памяти:  $O(|V|)$ . Максимальная длина пути равна  $|V| \Rightarrow$  выполняется  $\max_k k = |V|$ . Поскольку текущее состояние пересчёта опирается исключительно на предыдущее, достаточно хранить в памяти только две строки матрицы  $dp$ . Общее число обрабатываемых вершин в строке равно  $|V|$ .



## Алгоритм SPFA

- SPFA - **Shortest Path Faster Algorithm**
- Алгоритм находит кратчайшее расстояние в графе от одной вершины ко всем при отсутствии циклов отрицательного веса. Является статистически улучшенной версией алгоритма Форда-Беллмана.



## Алгоритм Флойда-Уоршелла

**Опр. Задача APSP (APSP, All Pairs Shortest Paths)** - задача поиска кратчайшего пути между любыми двумя вершинами графа.

### Цель

Алгоритм Флойда-Уоршелла предназначен для решения задачи APSP.

## Алгоритм

В основе работы лежит принцип динамического программирования. Занумеруем все вершины и зафиксируем нумерацию.

Пусть  $dp[u][v][k]$  - минимальный вес пути из  $u$  в  $v$ , т.ч. все промежуточные вершины имеют номера  $\leq k$ .

### 1. База:

- $\forall v \in V : dp[v][v][0] = 0$ .
- $\forall (u, v) \in E : dp[u][v][0] = weight_{u \rightarrow v}$ .
- Все остальные ячейки инициализируем  $+\infty$ .

### 2. Переход:

$$dp[u][v][k] = \min \{ dp[u][v][k-1], dp[u][k][k-1] + dp[k][v][k-1] \}$$

3. Порядок пересчёта: внешний цикл по  $k$ , внутренние циклы - по всем парам вершин  $u, v \in V$ .

4. Ответ:  $\forall u, v \in V : dp[u][v][|V|]$ .

## Корректность

### Доказательство

Докажем по индукции по параметру  $k$ .

**База:**  $k = 0$ . Промежуточные вершины отсутствуют. Значения инициализированы корректно.

**Предположение:**  $\forall u, v \in V$  значение  $dp[u][v][k-1]$  корректно задаёт минимальный вес пути из  $u$  в  $v$ , все промежуточные вершины которого имеют номера  $\leq k-1$ .

**Шаг:** Рассмотрим кратчайший путь  $P$  из  $u$  в  $v$ , промежуточные вершины которого принадлежат множеству  $\{1, \dots, k\}$ . Возможны два случая.

1. Пусть вершина  $k$  не является промежуточной для пути  $P \Rightarrow$  все промежуточные вершины имеют номера  $\leq k-1$ . По предположению индукции, вес такого пути равен  $dp[u][v][k-1]$ .
2. Пусть вершина  $k$  является промежуточной для пути  $P$ . Т.к.  $P$  - кратчайший путь и в графе отсутствуют циклы отрицательного веса, вершина  $k$  встречается на нём ровно один раз. Тогда путь разбивается на две части:  $u \rightsquigarrow k$  и  $k \rightsquigarrow v$ , промежуточные вершины которых имеют номера  $\leq k-1$ . По предположению индукции, их минимальные веса равны  $dp[u][k][k-1]$  и  $dp[k][v][k-1]$  соответственно  $\Rightarrow$  полный вес равен  $dp[u][k][k-1] + dp[k][v][k-1]$ .

## Асимптотика

- Сложность по времени:  $O(|V|^3)$ . На каждом из  $|V|$  шагов внешнего цикла полностью перебираются все пары вершин  $u, v \in V$ , количество которых составляет  $|V|^2$ . Каждая итерация пересчёта выполняется за  $O(1)$ .
- Сложность по памяти:  $O(|V|^2)$ . Формула перехода на шаге  $k$  опирается исключительно на слои предыдущего состояния  $k-1 \Rightarrow$  достаточно хранить в памяти две матрицы размера  $|V| \times |V|$ .



## Циклы отрицательного веса

### ⚠ Проблема

Если в графе  $\exists$  цикл отрицательного веса, то по нему можно перемещаться бесконечно, уменьшая вес пути в меньшую сторону. Найти такой цикл позволяют алгоритмы Форда-Беллмана и Флойда-Уоршелла.

## Алгоритм Форда-Беллмана для поиска цикла

Модифицируем стандартный алгоритм Форда-Беллмана, т.ч. для  $\forall v \in V$  дополнительно хранится её предок  $p[v]$ , из которого осуществлена последняя релаксация.

Алгоритм поиска и восстановления:

1. Запускается алгоритм Форда-Беллмана на  $|V|$  итераций.
2. Если на  $|V|$ -й итерации произошла релаксация рёбер  $\Rightarrow \exists$  путь, содержащий  $\geq |V|$  рёбер  $\Rightarrow$  в графе есть цикл.
3. Т.к. вес пути строго уменьшился  $\Rightarrow$  вес цикла отрицателен.
4. Нахождение вершины цикла: пусть релаксация на  $|V|$ -й итерации произошла для вершины  $v$ . Начиная с  $v$ , пройдем назад по предкам  $p[v]$  ровно  $|V|$  раз  $\Rightarrow$  гарантированно попадем в вершину  $x$ , лежащую на цикле отрицательного веса.
5. Восстановление цикла: двигаясь от вершины  $x$  назад по массиву предков  $p$  до повторного возвращения в  $x$ , получаем вершины цикла в обратном порядке.

## Алгоритм Флойда-Уоршелла для поиска цикла

Алгоритм Флойда-Уоршелла вычисляет кратчайшие расстояния между всеми парами вершин графа. Наличие цикла отрицательного веса определяется по значениям на главной диагонали результирующей матрицы динамического программирования  $dp$ .

**Критерий** Цикл отрицательного веса существует  $\iff \exists v \in V : dp[v][v][|V|] < 0$ .

## Доказательство



1. В базе динамического программирования значения на главной диагонали  $dp[v][v][0] = 0$ , т.к. путь из вершины в саму себя без промежуточных вершин имеет нулевую длину.
2. Промежуточные шаги алгоритма пересчитывают значения по формуле:
$$dp[u][w][k] = \min\{dp[u][w][k-1], dp[u][k][k-1] + dp[k][w][k-1]\}$$
3. Если для некоторой вершины  $v$  на шаге  $k = |V| : dp[v][v][|V|] < 0 \Rightarrow \exists$  нетривиальный путь из  $v$  в  $v$  отрицательного веса, что означает существование цикла отрицательного веса, проходящего через  $v$ .

Для восстановления самого цикла используется стандартная матрица предков  $next[u][v]$ , хранящая индекс первой промежуточной вершины на кратчайшем пути между  $u$  и  $v$ .

### 🔗 Применение на практике

Обнаружение циклов отрицательного веса имеет важное практическое применение в финансовых системах, например, на валютных биржах, в банковских межбанковских операциях и криптовалютных кошельках. Если представить обменные курсы как рёбра графа с весами, равными логарифму курса, то отрицательный цикл соответствует арбитражной возможности - последовательности обменов, при которой начальная сумма увеличивается за счёт разницы котировок. Алгоритмы Форда-Беллмана и Флойда-Уоршелла позволяют выявлять такие циклы и тем самым находить прибыльные арбитражные стратегии, а также своевременно обнаруживать аномалии в курсах, что критически важно для высокочастотной торговли и управления рисками.



## Алгоритм Джонсона

### 📌 Цель

Алгоритм Джонсона решает задачу APSP во взвешенном орграфе при условии отсутствия циклов отрицательного веса.

В рамках алгоритма введём понятия:

**Опр.** **Потенциал** - функция вида  $\varphi : V \rightarrow \mathbb{R}$ .

**Опр.** **Потенциальный вес ребра** - изменённый вес ребра, задаваемый как  $w_\varphi((u, v)) = weight_{u \rightarrow v} + \varphi(u) - \varphi(v)$ .

**Опр.** **Неотрицательный потенциал** - потенциал  $\varphi$ , для которого :  $\forall e \in E : w_\varphi(e) \geq 0$ .

### Лемма о сохранении кратчайших путей

- Если путь  $P$  являлся кратчайшим из  $s$  в  $t$  относительно исходных весов  $weight \Rightarrow \forall \varphi$  он остаётся кратчайшим относительно потенциальных весов  $w_\varphi$ .

### Доказательство

Для произвольного пути  $P = (p_1, p_2, \dots, p_n)$ , где  $p_1 = s, p_n = t$ , распишем его потенциальный вес:

$$w_\varphi(P) = \sum_{i=1}^n (weight_{p_i \rightarrow p_{i+1}} + \varphi(p_i) - \varphi(p_{i+1})) = \varphi(s) - \varphi(t) + \sum_{i=1}^n weight_{p_i \rightarrow p_{i+1}} = \varphi(s) - \varphi(t) + weight_P$$

Поскольку величина  $\varphi(s) - \varphi(t)$  константна  $\forall$  путей из  $s$  в  $t \Rightarrow$  веса всех путей изменяются на одну и ту же величину  $\Rightarrow$  кратчайший путь остаётся кратчайшим.

### Критерий существования неотрицательного потенциала

- $\exists$  неотрицательный потенциал  $\varphi$  для графа  $G \iff$  в  $G$  отсутствуют циклы отрицательного веса.

## Доказательство

**Необходимость:** Рассмотрим произвольный цикл  $C$  в графе  $G$ . Поскольку  $\forall e \in C : w_\varphi(e) \geq 0$  то  $weight_C = w_\varphi(C) \geq 0$ . Из этого следует неотрицательность любого цикла.

**Достаточность:** Добавим в граф новую фиктивную вершину  $s$  и проведём из неё рёбра во все остальные вершины с нулевыми весами. Зададим потенциал как  $\varphi(v) = dist(s, v) \forall v \in V$ . Тогда потенциальный вес  $\forall (u, v) \in E$  равен  $w_\varphi((u, v)) = \varphi(u) + weight_{u \rightarrow v} - \varphi(v) = dist(s, u) + weight_{u \rightarrow v} - dist(s, v)$ . По определению кратчайшего расстояния :  $dist(s, u) + weight_{u \rightarrow v} \geq dist(s, v) \Rightarrow w_\varphi((u, v)) \geq 0$ .

## Алгоритм Джонсона

1. Добавление фиктивной вершины  $s$  и запуск алгоритма Форда-Беллмана из неё для проверки отсутствия циклов отрицательного веса.
2. Вычисление потенциалов  $\varphi(v) = dist(s, v) \forall v \in V$  и пересчёт весов всех рёбер по формуле  $w_\varphi(e)$ .
3. Запуск алгоритма Дейкстры (или  $A^*$ ) из каждой вершины  $v \in V$  на графе с новыми неотрицательными весами рёбер.

## Корректность

Корректность следует из леммы о сохранении кратчайших путей и критерия существования неотрицательного потенциала. Т.к. после изменения весов рёбер на  $w_\varphi(e) \geq 0$  множества кратчайших путей не изменились  $\Rightarrow$  запуск алгоритма Дейкстры из каждой вершины корректно найдёт исходные кратчайшие расстояния.

## Асимптотика

- Сложность по времени:  $O(|V| \cdot |E| \log |V|)$ . Складывается из запуска алгоритма Форда-Беллмана за  $O(|V| \cdot |E|)$ , пересчёта потенциалов рёбер за  $O(|V| + |E|)$  и  $|V|$  запусков алгоритма Дейкстры на двоичной куче, что занимает  $|V| \cdot O(|E| \log |V|) = O(|V||E| \log |V|)$ .
- Сложность по памяти:  $O(|V| + |E|)$  для хранения скорректированных весов графа и результатов работы алгоритмов.



## Двусторонние алгоритмы

Двусторонние алгоритмы запускаются сразу из двух мест: из начальной вершины и целевой вершины. Главная причина использования двусторонних алгоритмов - **экспоненциальное сокращение поискового пространства** (количества фактически посещённых и раскрытых вершин) на реальных данных. В задачах маршрутизации (например, карты, GPS-навигаторы) графы огромны, и обычные алгоритмы работают слишком медленно.

## Двусторонний BFS

### Цель

Двусторонний обход в ширину (Bidirectional BFS) находит кратчайшее расстояние между заданными вершинами  $s$  и  $t$  во взвешенном графе при условии, что **все рёбра имеют одинаковый вес**.

## Алгоритм

Параллельное подведение двух встречных фронтов поиска:

1. Запускается обход  $BFS$  параллельно из стартовой вершины  $s$  по исходному графу  $G$  и из целевой вершины  $t$  по транспонированному графу  $G^T$ .
2. Поиск завершается, когда фронт обхода из вершины  $s$  добавляет в очередь вершину  $u$ , уже посещённую фронтом обхода из вершины  $t$  (или наоборот).

3. Ответ: если общая встреченная вершина -  $u \Rightarrow dist(s, t) = d[u] + d^T[u]$ .

## Двусторонний алгоритм Дейкстры

### Цель

Двусторонний алгоритм Дейкстры находит кратчайшее расстояние между вершинами  $s$  и  $t$  в графе с неотрицательными весами рёбер.

### Алгоритм

1. Запускается алгоритм Дейкстры параллельно из вершины  $s$  по исходному графу  $G$  и из вершины  $t$  по транспонированному графу  $G^T$ .
2. Пусть  $S$  - множество вершин, для которых кратчайшее расстояние рассчитано прямым алгоритмом Дейкстры, а  $S^T$  - множество вершин, обработанных транспонированным алгоритмом.
3. Алгоритм останавливается в момент, когда некоторая вершина  $m$  извлекается из очередей обоих обходов одновременно ( $m \in S \cap S^T$ ).
4. Ответ вычисляется как минимум среди всех путей, проходящих через рёбра между посещёнными множествами:

$$dist(s, t) = \min \left\{ dist(s, m) + dist(m, t), \min_{u \in S, v \in S^T} (dist(s, u) + weight_{u \rightarrow v} + dist(v, t)) \right\}$$

## Двусторонняя A\*

### Цель

Двусторонний алгоритм  $A^*$  представляет собой эвристическую модификацию встречного поиска кратчайшего пути от  $s$  до  $t$  во взвешенном графе с неотрицательными весами рёбер.

### Проблема

Прямой независимый запуск двустороннего алгоритма  $A^*$  невозможен, т.к. эвристические оценки для прямого и обратного направлений будут несогласованными. Требуется приведение потенциалов к согласованному виду.

Доработаем эвристические функции. Пусть  $h_s(v)$  - эвристика для оценки расстояния  $dist(s, v)$ , а  $h_t(v)$  - эвристика для оценки расстояния  $dist(v, t)$ .

**Опр. Согласованность двусторонних эвристик** - условие, для которого  $\forall u, v \in V$ :

$$h_s(u) + h_t(u) = h_s(v) + h_t(v) = const.$$

Для выполнения условия пересчитаем эвристики по формуле:

$$h'_s(v) = \frac{h_s(v) - h_t(v)}{2} = -h'_t(v)$$

Модифицируем веса рёбер графа, т.ч. эвристическая функция влияет на вес перехода по ребру:

$$weight_{u \rightarrow v}^h = weight_{u \rightarrow v} - h(u) + h(v)$$

Данная модификация применяется к графу  $G$  с эвристикой  $h := h'_s$  и к транспонированному графу  $G^T$  с эвристикой  $h := h'_t$ .

### Алгоритм

1. Запускается параллельный эвристический поиск на изменённых графах  $G$  и  $G^T$ .

2. Пусть  $S$  - множество вершин, обработанных прямой версией  $A^*$ , а  $S^T$  - множество вершин, обработанных транспонированной версией.

3. При встрече алгоритмов в вершине  $m$  ответ вычисляется по формуле:

$$dist(s, t) = \min \left\{ dist(s, m) + dist(m, t), \min_{u \in S, v \in S^T} (dist(s, u) + weight_{u \rightarrow v} + dist(v, t)) \right\}$$



Разрыв страницы

## Примечания

1. Оригинальный конспект в последней редакции можно найти на моём GitHub: <https://github.com/Leg15Coder/Digital-garden>
2. По всем найденным ошибкам и опечаткам можно писать в GH issues: <https://github.com/Leg15Coder/Digital-garden/issues> или мне в ЛС [https://t.me/dimka\\_ryaz](https://t.me/dimka_ryaz)